

Terrain Generation Using Perlin Noise and Erosion Simulation

Introduction

The goal of this project was to develop a terrain generation tool that allows users to create and modify a 3D mesh-based terrain. The tool allows users to generate a terrain mesh with customizable resolution and to modify them using Perlin noise. Additionally, it features a particle-based hydraulic erosion technique that simulates the effects of water flow. User can create a variety of diverse terrains by adjusting different parameters and applying erosion to the mesh.

Accomplishments

The project was able to accomplish multiple goals.

Mesh-based Terrain with Customizable Resolution

The customizable resolution of the terrain allowed users to decrease the number of vertices if needed. This allows for flexibility in the terrain's level of detail and showcases how the noise affects different levels of terrain complexity.

Procedural Terrain Generation Using Perlin Noise

The integration of Perlin noise combined with fractal Brownian motion allowed for the generation of natural-looking terrains with the ability to tweak parameters such as scale, persistence, lacunarity and the number of octaves to control the terrain shape.

Particle-based Hydraulic Erosion

This system simulates how water droplets interact with the terrain or mesh by following gradients, picking up and depositing sediment, and modifying the landscape overtime. As the “droplets” move, they are able to erode steep slopes and deposit material in lower region, simulating real-life sediment transport. This technique added an extra layer of realism and variety to the terrain by making them more organic and less uniform than simple noise-generated terrain.

Functional Interface Allowing Modification of Terrain Properties

The interface allows users to tweak parameters and observe the resulting changes. This feature makes the tool user friendly and useful for quick experimentation and generation of random terrain.

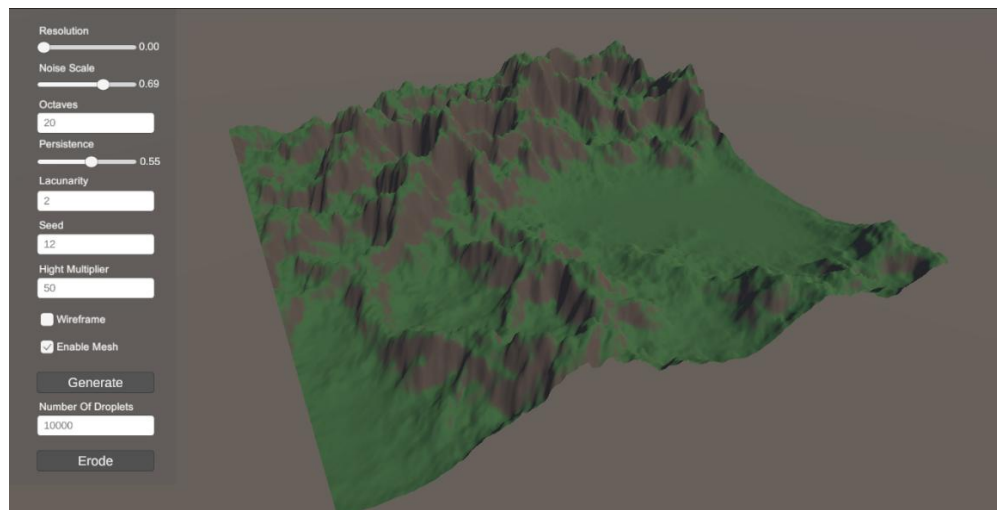


Figure 1: Example of a generated terrain with noise and erosion.

Key Algorithms Implemented

Perlin Noise with Fractal Brownian Motion

The Perlin Noise function was implemented using a permutation-based noise function. It works by assigning random gradient vectors to grid points and interpolating values between them to create smooth variations. The random vectors are selected from a set of predefined constant vectors to determine the direction of gradients at each point, ensuring consistency in the noise pattern.

To enhance the terrain's realism and make it less smooth, fractal Brownian motion (FBM) was applied. This allowed the layering of multiple octaves of Perlin noise at different frequencies and amplitudes, allowing the terrain to look a bit more organic than simple random noise.

Erosion Algorithm

The hydraulic erosion algorithm was implemented using a particle-based simulation technique. The process starts by creating water droplets at random positions on the terrain. Each droplet follows the current height gradients and adjusts its direction and speed based on the terrain's slope. As the droplet moves it either picks up or deposits sediment, modifying the terrain. The sediment capacity is dynamically adjusted based on speed and slope steepness, allowing a realistic simulation of erosion.

The erosion technique uses bilinear interpolation to distribute sediment across the terrain grid. When a droplet deposits sediment, the material is distributed among the surrounding grid points based on the droplet's exact location. This prevents sharp or unnatural edges in the terrain. Additionally, erosion is applied using a weighted brush effect, ensuring that the elevation changes gradually rather than in abrupt cuts.

To refine the erosion effect more, parameters like inertia, friction and evaporation rate were added. Inertia determines how much a droplet resists changing direction, while friction and evaporation affect the droplet's lifespan and ability to transport sediment.

Challenges

Despite the successes a few challenges were encountered during development:

Parameter Fine-Tuning

Finding the right balance of parameters for erosion was difficult, as small adjustments could lead to unrealistic or undesirable terrain formations. After some experimentation, I found that blending the eroded terrain with the original terrain offered the best solution, preserving some landscape features while still benefiting from the natural-looking erosion effects. This approach was able to provide better stability and control over the final outcome compared to solely relying on perfect parameter settings.

Performance Optimization

The erosion algorithm, while effective, was computationally expensive, especially for high numbers of particles. Implementing optimizations such as parallel processing or a more efficient data structure could improve performance.

Future Work

Some features that weren't implemented due to time constraints include an interface for adjusting additional hydraulic erosion parameters. This would give the user even more control over the final result. Additionally, the

particle-based erosion simulation tends to be quite computationally expensive especially when using a very large amount of particles, so further optimization was considered but not implemented due to the same reason.

Future enhancements could also include functionality to save and export the generated mesh and support for additional erosion types and noise generators.

Conclusion

During this project, I was able to develop a working terrain generation tool with pseudorandom noise and erosion effects. While there still are certain features that could be improved, the tool is a strong foundation for further development.

This project provided me with valuable insights into procedural terrain generation, pseudorandom noise functions, and water erosion simulations, allowing me to gain a deeper understanding of mathematical modeling.

References

McDonald, N. (2020, April 10). Simple particle-based hydraulic erosion. Retrieved May 31, 2025, from <https://nickmcd.me/2020/04/10/simple-particle-based-hydraulic-erosion/>

Musgrave, F., Kolb, C., & Mace, R. (1998). The synthesis and rendering of eroded fractal terrains. *Computer Graphics Forum*, 9(4), 185–194. [https://doi.org/10.1002/\(SICI\)1099-1778\(1998100\)9:4%3C185::AID-VIS178%3E3.0.CO;2-2](https://doi.org/10.1002/(SICI)1099-1778(1998100)9:4%3C185::AID-VIS178%3E3.0.CO;2-2)

Rtouti, R. (n.d.). Perlin noise algorithm. Retrieved May 15, 2025, from <https://rtouti.github.io/graphics/perlin-noise-algorithm>

Talle, J. (n.d.). Simulating hydraulic erosion. Retrieved May 24, 2025, from https://jobtalle.com/simulating_hydraulic_erosion.html

University of Illinois Urbana-Champaign. (n.d.). Erosion simulation. Retrieved May 29, 2025, from <https://cs418.cs.illinois.edu/website/text/erosion.html>

Digital Free Pen. (2017, June 20). Range Perlin noise. Retrieved May 20, 2025, from <https://digitalfreepen.com/2017/06/20/range-perlin-noise.html>